

Novel FPGA based Systolic Array Design for Dynamic Workflows

Aimy Jose

Department of Electronics Engineering
Model Engineering College, Thrikkakara
APJ Abdul Kalam Technological University
Kerala, India
aimy jose.mec@gmail.com

Jobymol Jacob

Department of Electronics Engineering
Model Engineering College, Thrikkakara
APJ Abdul Kalam Technological University
Kerala, India
joby@mec.ac.in

Abstract—Convolutional neural network based object detection is an inevitable part of advanced driver assistance systems which has high accuracy and low latency requirements. Considering the high computational complexity of CNNs, it is essential to have customized hardware to boost their performance. In this regard it is essential to implement an optimized CNN algorithm on FPGA. The high energy efficiency, parallel computing capabilities and reconfigurability of FPGAs makes it suitable for hardware acceleration of CNNs. In this work a modified dynamic systolic array based CNN accelerator is implemented targeting PYNQ Z2 FPGA board where the most suitable architecture among the three typical systolic array architectures is selected based on the workload. Here, the processing element of systolic array is designed using brain floating point 16 bit format to accommodate better data range as compared to floating point 16 bit format. The results of the model with respect to floating point operations are analysed.

Index Terms—FPGA accelerator, Convolutional Neural Networks, Systolic array, Dynamic workflows, bfloat16.

I. INTRODUCTION

In the ever-evolving landscape of advanced driver assistance systems (ADAS), the integration of cutting-edge technologies is crucial for enhancing the safety and efficiency of modern vehicles. Convolutional neural networks (CNNs) play a pivotal role in ADAS applications, especially for tasks like object detection, lane detection, and traffic sign recognition. To meet the real-time processing demands of these computationally intensive tasks, there is a growing trend towards leveraging field-programmable gate arrays (FPGAs) as accelerators for CNN workloads.

CNNs have emerged as the solution for computer vision tasks within ADAS. They excel at extracting hierarchical features from image data, enabling the identification of objects, pedestrians, traffic signs and road markings. The accuracy and efficiency of CNNs make them indispensable for enhancing the perception capabilities of vehicles and enabling adaptive responses to dynamic driving scenarios.

The hardware platforms that are prominently used for CNN accelerators are graphical processing units (GPU), application specific integrated circuits (ASIC) and field programmable gate arrays (FPGA). Since, power efficiency is a critical consideration in the automotive domain, FPGAs, through

their ability to optimize hardware resources for specific CNN models and tasks, offer a balance between computational performance and power consumption. This is particularly relevant for ADAS, where energy efficient solutions are vital for on-board systems. FPGAs also offer a unique combination of flexibility, integration capability, parallel processing capabilities and high reconfigurability, making them well-suited for the dynamic ADAS applications that constantly evolve to address the challenges of real-time image processing and decision-making in automotive environments.

In case of time critical [1] ADAS systems, the electronic control unit (ECU) has to generate signals to control braking, steering and throttling action of the vehicle in real-time, based on the output coming from the CNNs that perform object detection (OD). As the applications becomes complex, the architecture of the CNN becomes more complex as the depth of the network increases to meet the number of computations that has to be handled. Convolution layer that are computation intensive and the fully connected layer that is memory-intensive adds to the bottleneck during the deployment of the model [2], [3].

This makes it challenging to deploy CNNs in real-time ADAS applications as these have constraints in terms of response time, memory requirements, resource utilization and power consumption. It is therefore essential to search for an efficient and optimised design for CNN accelerator [4].

J. Zhang et. al. [5] focused on the frequency improvement of systolic array based CNNs on FPGA. They identified the reasons for the frequency degradation of systolic array designs for CNN accelerators and propose methods to improve the frequency at the front-end and the back-end. H. Ahmed et.al. [6] proposed a design space exploration methodology based on an analytical model that was formulated to estimate the required resources for systolic arrays, assuming multiple data reuse pattern. They ported convolutional layers of TINY YOLO on a Xilinx Artix 7 FPGA. L. Jia et. al. [7] built hardware module templates using Chisel infrastructure that could be reused for different data flows and computation algorithms unlike the existing High Level Synthesis (HLS) tools that do not favor modular and reusable design.

In this paper, a modified systolic array based CNN accelerator

ator is designed targeting PYNQ Z2 FPGA for object detection in ADAS applications. A state-of-the-art neural network design based on different systolic array architectures: output stationary (OS), weight stationary (WS), and input stationary (IS).

In this design an innovative systolic array processor is developed featuring dynamic dataflows, allowing it to adapt to different workloads and consistently achieve optimal performance. To further enhance efficiency, our implementation leverages brain floating point 16 (bfloat16), optimizing the hardware utilization and improving accuracy. A detailed performance comparison of the systolic array architectures across various computing environments is performed.

The paper is organized as follows. Section II discusses the underlying principles of systolic array based CNN and its architecture. Section III deals with the design of the model targeting PYNQ Z2 FPGA. In section IV, the experimental results are analyzed and performance evaluation is done. Section V concludes the work.

II. SYSTOLIC ARRAY BASED CNN MODEL

This section discusses the use of the Systolic array based CNN model for object detection.

A. Systolic Array

A systolic array is characterized by a grid of processing elements arranged in rows and columns. Each processing element performs a specific computation and passes its result to its neighboring elements. In CNNs, the convolution operation involves sliding a filter over the input data, performing element-wise multiplications, and summing the results to produce the output feature map. This operation is computationally intensive, especially for large input data and complex network architectures [8]. Systolic arrays are excellent at exploiting parallelism, with each processing element concurrently handling different portions of the input data. The parallel nature of systolic arrays is well-suited for the matrix multiplication and summation steps involved in convolution operations. In addition, data flows through the systolic array in a pipelined manner which allows for continuous and efficient computation without the need for frequent data transfers.

Systolic arrays typically utilize specific dataflows to manage data movement across processing elements (PEs). Each dataflow ensures that one type of data—input, output, or weights—remains stationary for the maximum duration to maximize data reuse [9]. The most common systolic array dataflows include:

- **Weight stationary (WS):** Weights are preloaded into the PEs and remain stationary throughout computation, while the input matrix and partial sums move between PEs [10]. This dataflow is widely adopted due to its scalability, low memory requirements and efficiency in handling convolutions and matrix multiplications. As shown in Fig 1. For a 8x8 systolic array, weight stationary workflow will be considered in the case where the dimension of weight matrix is

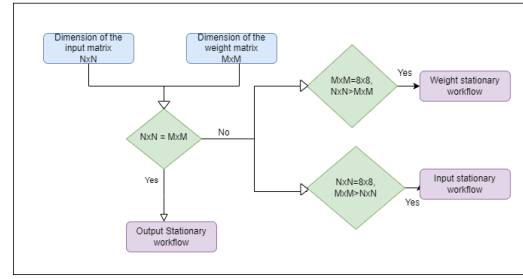


Fig. 1. Flow Chart

equal to 8x8 and that of input matrix is greater than 8x8. In this case the input matrix has to be loaded in parts while the weight matrix remains the same all throughout.

- **Input Stationary (IS):** The input matrix is initially loaded into the systolic array, while weights and partial summations move through the PEs during processing [11]. This is selected if the dimension of input matrix is equal to 8x8 and that of weight matrix is greater than 8x8. Here the weight matrix is loaded in parts while the input matrix remains the same.
- **Output Stationary (OS):** Both input and weight matrices move across the PEs, while partial sums are accumulated within each PE. However, this dataflow effectively doubles memory bandwidth requirements [12], [13], [14]. OS dataflow is selected in the case where the dimension of both input and weight matrix are the same.

However, regardless of the type of systolic array used, each dataflow has scenarios where it does not achieve optimal performance. To address this limitation, this paper introduces a novel bfloat16 based systolic array architecture with dynamic data flows, enabling the processor to adapt its data flow based on the workload and consistently achieve the best performance in any situation.

B. Bfloat16 based Processing Elements

The PEs are the fundamental computational units in a systolic array, responsible for executing matrix operations efficiently. Each PE in the systolic array is responsible for computing the weighted sum of inputs. It performs the multiplication of input values by corresponding weights and accumulates the results. To enhance performance and energy efficiency, bfloat16 arithmetic, as opposed to floating point 16 (fp16) or floating point 32 (fp32), is incorporated in the design.

Usage of bfloat16 arithmetic in PEs has the following advantages:

- **Reduced bit width:** As represented in fig. 2, bfloat16 uses 16 bits-1 sign bit, 8 exponent bits (same as fp32), and 7 mantissa bits. This preserves the dynamic range of fp32 while reducing computational complexity.

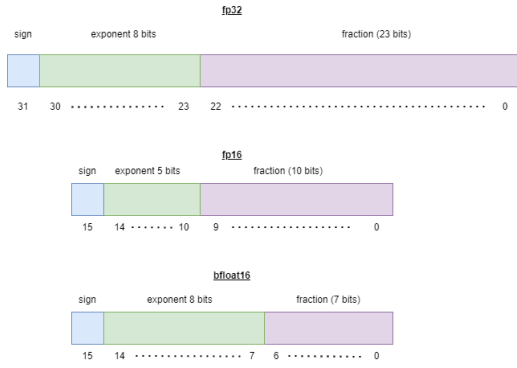


Fig. 2. Comparison of bfloat16 with IEEE 754 single-precision (fp32) and half-precision (fp16)

- **Low Memory Bandwidth and Energy Efficiency:** Compared to fp32, bfloat16 reduces memory access, improving energy efficiency and throughput.
- **Optimized multiply-accumulate (MAC) Operations:** The PE utilizes bfloat16-based MAC units to accelerate matrix multiplications which is a core operation in deep learning workloads.

In addition to this bfloat16 has a range of $\sim 1.1e-38$ - $\sim 3.4e38$ which is same as that of fp32 unlike fp16 that only has a range of $\sim 5.96e-8$ - $\sim 6.5e4$ which allows it to represent much larger and smaller numbers than fp16, making it better suited for tasks where numerical stability and range are more critical. This means that bfloat16 can be used in place of fp32 when training and running deep neural networks with less risk of overflow or underflow compared to FP16, while still reducing memory usage and computational load. Additionally, this is achieved without much loss in precision unlike fp16 which requires techniques such as loss scaling to compensate for the loss of precision.

Each bfloat16-based PE operates within the systolic array using the following components:

- **MAC Unit:** Performs multiply-accumulate operations in bfloat16 format.
- **Local Buffer:** Stores stationary data (weights, inputs, or partial sums) depending on the selected dataflow.
- **Data Routing Logic:** Manages data movement between PEs based on the chosen systolic dataflow architecture (WS, IS, OS or dynamic DD).

By leveraging bfloat16-based PEs, systolic array achieves high computational efficiency, reduced memory bandwidth usage, and optimal adaptability across different workloads.

Fig. 3. represents the design of a single processing element which includes a weight preload for weight matrix, input preload for input matrix, a bfloat16 multiplier where input and weight multiplication happens, a bfloat32 adder which is responsible for the addition of partial sum.

Based on the workload the two control signals select whether the architecture employed is WS, IS, or OS. In the case of WS, the input/weight stationary control signal is set in such a way that the weights are preloaded to the PE. During

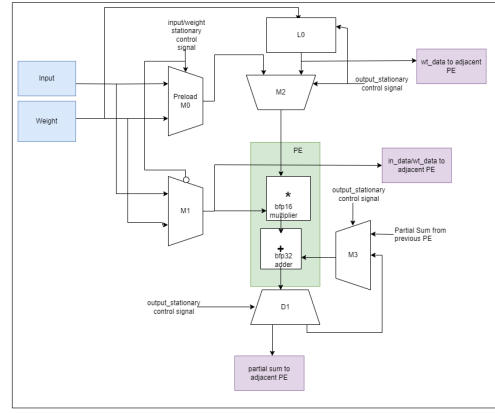


Fig. 3. Processing Element Design

the computation, the inputs are moved through the array, and at each step, they are multiplied by the corresponding fixed weights. In the case of IS, inputs get preloaded while the weights move through the array. When o_s_control signal is set both weight and input moves through the array while the partial sum gets accumulated.

In IS and WS workloads all the weight/input data are preloaded together, so the time taken for the weights/inputs to cascade down the PE array reduces significantly. The weights and input data are converted to floating point representations to analyse the latency.

III. DESIGN AND IMPLEMENTATION OF SYSTOLIC ARRAY FOR DYNAMIC WORKLOADS

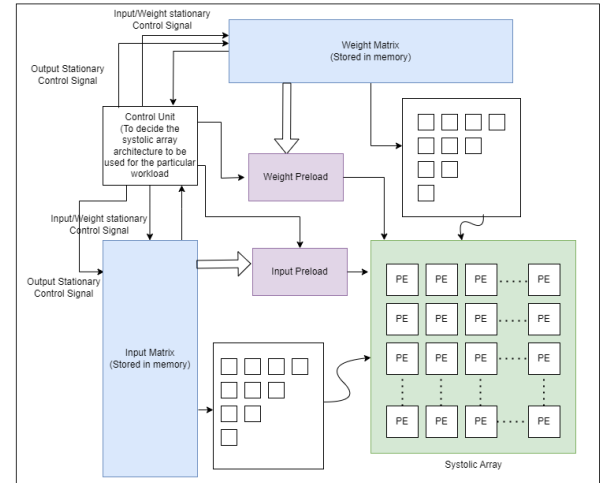


Fig. 4. Dynamic Systolic Array

The optimal dataflow can be determined based on the sizes of the input and weight matrices. As in Fig. 4. in the dynamic systolic array architecture, prior to computation, the system generates control signals tailored to the sizes of the input and weight matrices. When data is input, these control signals are

sent to the systolic array, directing which dataflow to use for the calculation. When the input and weight matrices are of equal size and entire computation fits in the systolic array, the control signal is configured as OS to enhance performance. In this case both input and weight values gets streamed into the processing element until the entire matrices are covered. If both matrices are of dimension $M \times M$ (where $n = 2, 3, 4 \dots$ etc.) and of equal size, the control signal is set to IS or WS. If the dimension of the weight matrix is such that it fits in the systolic array, the control signal is configured as WS so that the weights need to be loaded only once; while the input matrix keeps getting loaded according to the matrix size. If the dimension of the input matrix is such that it fits in the systolic array while weight matrix has to keep getting loaded, the control signal is set to IS. The common features across the three dataflows simplify the implementation of the dynamic systolic array. Depending on the chosen dataflow, the data processing and transmission within the matrix multiply unit vary. The MMU in the dynamic systolic array integrates the functions of all three typical systolic arrays, resulting in a significant increase in hardware overhead. However, the number and configuration of registers, multipliers, and adders across the three standard systolic arrays are similar, allowing for the reuse of these components in the design.

The design for the systolic array is capable of multiplying two parameterized $N \times M$ matrices using brain floating-point values. Bfloat16 has the same exponent range as fp32 but with a reduced precision in the mantissa. This allows it to represent much larger and smaller numbers than fp16, making it better suited for tasks where numerical stability and range are more critical. Bfloat16 can handle the large range of values encountered during training with less risk of overflow or underflow compared to fp16, while still reducing memory usage and computational load. Bfloat16 is more compatible with fp32, as it preserves the same exponent size. This makes it easier to convert between fp32 and bfloat16 with less risk of losing information, which is valuable during training and inference phases of machine learning.

IV. RESULT ANALYSIS

In this section, the experimental results obtained are analyzed and the performance of the proposed model is evaluated in terms of the metrics such as efficiency, resource utilization and latency.



Fig. 5. Simulation result of a bfloat16 Processing Element

Fig.5 represents the bfloat16 multiplier for two 15 bit inputs A and B for various cases. It analyses the corner cases associated with floating point multiplication such as multiplication when one or both of the inputs is NaN (not a

number). In such a case it can be seen that the result obtained is also NaN. If either A or B is 0 the product obtained is 0 and if a or b is inf (infinity) return inf. It can also be seen from the simulation results that multiplication of two positive numbers resulted in a positive number while the result of multiplication of a positive and negative number resulted in a negative product. The simulation result also includes the multiplication of two fractional numbers.

	Slice LUTs (53200)	Slice Registers (106400)	DSPs (220)	Bonded IOBs (125)
bfloat16	156	21	0	49
fp32	115	21	2	97

Fig. 6. Comparison of resource utilization of bfloat16 vs fp32 multipliers

	Dynamic	Device static	Total power consumption
bfloat16	10.752W	1.039W	11.791W
fp32	13.665W	1.039W	14.704 W

Fig. 7. Comparison of power consumption of bfloat16 vs fp32 multipliers

As it can be seen from Fig. 6 and Fig. 7, high level of energy efficiency can be achieved with lesser number of bits by using bfloat16.

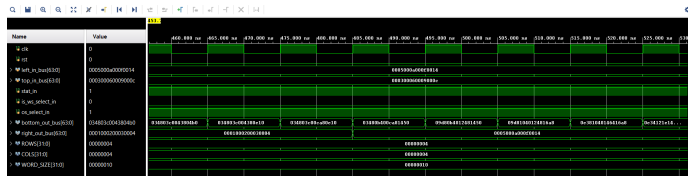


Fig. 8. Simulation result of the modified systolic array design (i)



Fig. 9. Simulation result of the modified systolic array design (ii)

Fig. 8 and 9. represents the simulation result obtained from the implementation of the design. It shows how the result changes step by step as the input given to the systolic array changes.

Fig 10. depicts the resource utilization of the modified systolic array for various array dimensions. It represents the resources utilized, when the model is designed for PYNQ Z2 FPGA. It shows that for a 16x16 systolic array the proposed prototype occupies 33.86% of LUTs, 59.47% of LUTRAMs, and 100% of DSP. It can be seen that as the dimension of the matrix doubles the usage of LUTs and LUTRAMs only increases approx 25%.

Systolic array size	Slice LUTs (53200)	Slice Registers (106400)	DSPs (220)	BUFGCTRL (32)
4x4	768	772	16	1
8x8	3072	3136	64	1
16x16	18012	13084	220	1

Fig. 10. Resource utilization

Systolic array size	Slice LUTs (53200)	Slice Registers (106400)	DSPs (220)	BUFGCTRL (32)
Proposed 8x8	3072	3136	64	1
8x8 ws traditional	2825	2948	-	1

Fig. 11. Comparison of resource utilization of proposed bfloat16 systolic for dynamic workflows vs traditional 8x8 WS systolic array

Fig. 11 compares the resource utilization of the proposed bfloat16 systolic array for dynamic workflows with traditional 8x8 WS systolic array. It can be seen that the resource utilization of the two are comparable with the added advantage that the proposed systolic array can work with IS, WS, and OS workflows.

V. CONCLUSION

This paper presents a novel systolic array architecture utilizing bfloat16 arithmetic for a dynamic dataflow approach. By integrating the benefits of OS, WS, and IS dataflows, the proposed design can adaptively switch between dataflow methods based on matrix sizes. Experimental results demonstrate that this approach has achieved better performance with slight increase in the resource utilization compared to the traditional individual workflows. This design can also be optimized for mixed-precision workflow during CNN inference, where the best precision choice for balance between computational efficiency and numerical stability can be achieved.

ACKNOWLEDGMENT

The authors would like to sincerely thank the Chips to Startup (C2S) program of the Ministry of Electronics and Information Technology (MeitY) for providing necessary resources to carry out this work. The authors also acknowledge the Ministry of Minority Affairs (MoMA) for the financial support extended through the Maulana Azad National Fellowship Scheme (MANF) to carry out this work.

REFERENCES

- [1] X. Wang, K. Huang and A. Knoll, "Performance Optimisation of Parallelized ADAS Applications in FPGA-GPU Heterogeneous Systems: A Case Study With Lane Detection," in *IEEE Transactions on Intelligent Vehicles*, vol. 4, no. 4, pp. 519-531, Dec. 2019, doi: 10.1109/TIV.2019.2938092.
- [2] J. Kim, J. -K. Kang and Y. Kim, "A Low-Cost Fully Integer-Based CNN Accelerator on FPGA for Real-Time Traffic Sign Recognition," in *IEEE Access*, vol. 10, pp. 84626-84634, 2022, doi: 10.1109/ACCESS.2022.3197906.
- [3] T. Abtahi, C. Shea, A. Kulkarni and T. Mohsenin, "Accelerating Convolutional Neural Network With FFT on Embedded Hardware," in *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 26, no. 9, pp. 1737-1749, Sept. 2018, doi: 10.1109/TVLSI.2018.2825145.

- [4] M. Hailesellase, S. R. Hasan, F. Khalid, F. A. Wad and M. Shafique, "FPGA-Based Convolutional Neural Network Architecture with Reduced Parameter Requirements," *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*, Florence, Italy, 2018, pp. 1-5, doi: 10.1109/ISCAS.2018.8351283.
- [5] J. Zhang, W. Zhang, G. Luo, X. Wei, Y. Liang and J. Cong, "Frequency Improvement of Systolic Array-Based CNNs on FPGAs," *2019 IEEE International Symposium on Circuits and Systems (ISCAS)*, Sapporo, Japan, 2019, pp. 1-4, doi: 10.1109/ISCAS.2019.8702071.
- [6] Ahmad, Hazaar, et al. "Systimator: A Design Space Exploration Methodology for Systolic Array based CNNs Acceleration on the FPGA-based Edge Nodes." *arXiv e-prints* (2018): arXiv-1901.
- [7] L. Jia, L. Lu, X. Wei and Y. Liang, "Generating Systolic Array Accelerators With Reusable Blocks," in *IEEE Micro*, vol. 40, no. 4, pp. 85-92, 1 July-Aug. 2020, doi: 10.1109/MM.2020.2997611.
- [8] Ciylan, Furkan, Bünyamin Ciylan, and Mehmet Atak. "FPGA-Based Chaotic Image Encryption Using Systolic Arrays," *Electronics* (2079-9292), (2023).
- [9] Bo Wang, Sheng Ma, Guoyi Zhu, Xiao Yi, Rui Xu, A novel systolic array processor with dynamic dataflows, *Integration*, Volume 85, 2022, Pages 42-47, ISSN 0167-9260, <https://doi.org/10.1016/j.vlsi.2022.03.002>.
- [10] Xu, Rui & Ma, Sheng & Wang, Yaohua & Chen, Xinhai & Guo, Yang. (2021). Configurable Multi-directional Systolic Array Architecture for Convolutional Neural Networks. *ACM Transactions on Architecture and Code Optimization*. 18. 1-24. 10.1145/3460776.
- [11] B. Kim, S. Li and H. Li, "INCA: Input-stationary Dataflow at Outside-the-box Thinking about Deep Learning Accelerators," *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, Montreal, QC, Canada, 2023, pp. 29-41, doi: 10.1109/HPCA56546.2023.10070992.
- [12] H. Wan, C. Rao, Y. Zheng, P. Zhou and X. Lou, "A Systolic Array with Activation Stationary Dataflow for Deep Fully-Connected Networks," *2023 IEEE 5th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, Hangzhou, China, 2023, pp. 1-5, doi: 10.1109/AICAS57966.2023.10168602.
- [13] Inayat, Kashif, and Jaeyong Chung. "Hybrid accumulator factored systolic array for machine learning acceleration." *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 30.7 (2022): 881- 892.
- [14] See, Jin-Chuan & Ng, Hui-Fuang & Tan, Hung-Khoo & Chang, Jing-Jing & Mok, Kai-Ming & Lee, Wai-Kong & Lin, Chih-Yang. "Cryptensor: A Resource-Shared Co-Processor to Accelerate Convolutional Neural Network and Polynomial Convolution," in *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 12, pp. 4735-4748, Dec. 2023, doi: 10.1109/TCAD.2023.3296375.
- [15] Shibo Wang and Pankaj Kanwar, "BFloat16: The secret to high performance on Cloud TPUs", [Online]. Available: <https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus>.